

Air Hockey Lab · Chat Log

Date: 2026-10-01

Tool Used: Google Antigravity (Claude Opus 4.6)

Prompt 1: Read the Codebase

User:

context: read the code base end to end Instructions:

- 1 Go to the repo assigned to you
- 2 Go through the ReadMe file to understand the deliverables
- 3 Clone or Fork the repo
- 4 Run the Python based code (A game/ A Puzzle)
- 5 Record 10 seconds of video before making any changes.
- 6 Write prompts and fix the broken code
- 7 Write prompts to add all the features listed in the ReadMe file of the repo one by one.
- 8 Record another 10 seconds of video again after making all the changes.
- 9 Push the code to your repo.
- 10 Have Separate Commits for each task dont make changes now i will tell you when to do

Assistant:

I explored the entire repository structure and read all files end to end. Here is the analysis:

Files (7 source files + 2 config files)

| File | Purpose | |-----|-----| | `main.py` | Entry point · pygame init, game loop at 60 FPS | | `game_engine.py` | Core game logic · owns puck, paddles, AI, handles input/update/draw | | `puck.py` | Puck class · position, velocity, movement, wall bounce | | `paddle.py` | Paddle class · position, clamping to half-table, movement | | `collisions.py` | Puck·paddle collision handling **has the bug** | | `ai.py` | Computer AI · tracks puck Y with reaction delay & imprecision | | `renderer.py` | All drawing · table, walls, goals, puck, paddles, text/banners |

4 Tasks from README

Task 1: Fix puck-paddle collision (Bug)

Location: `collisions.py`

The bug: When the puck collides with a paddle, the code only reverses vx. This is wrong because:

- 1 It doesn't calculate the proper reflection angle · it ignores the collision normal. The puck should bounce based on the angle of impact, not just flip horizontally.
- 2 It doesn't push the puck outside the paddle after collision, so at high speeds the puck can get stuck inside the paddle, causing repeated vx flips (vibrating/stuck behavior).

Task 2: Implement match scoring (Missing feature)

What's missing: No score variables exist anywhere. When a goal is scored in `_handle_goals()`, it just resets the puck without updating any score. No score is displayed on screen.

Task 3: Implement 30-second match timer (Missing feature)

What's missing: No timer/countdown exists. The game runs indefinitely. No end-of-match logic or winner determination.

Task 4: Implement puck reset after scoring (Missing feature)

Location: `_reset_puck()`

What's wrong: The reset method moves the puck to center but sets velocity to $(0, 0)$ · the puck just sits still and never moves again. It should re-launch the puck with a random velocity.

Prompt 2: Task 1 · Fix Puck·Paddle Collision

User:

lets start with task1

Assistant:

Fixed the collision bug in `collisions.py`.

What was broken

The old code just did `puck.vx = -puck.vx` · a simple horizontal flip. This caused:

- **Wrong bounce angles** · the puck always bounced as if hitting a vertical wall, regardless of where on the paddle it hit
- **Puck getting stuck** · at high speeds the puck could clip inside the paddle, causing repeated vx flips (vibrating/stuck)

What was fixed

Replaced the naive vx-flip with proper angle-based reflection:

```
def handle_paddle_collision(puck, paddle):
    """
    If the puck overlaps the paddle, reflect its velocity along the
    collision normal (the line connecting the paddle centre to the
    puck
    centre) and push the puck out so it never gets stuck inside.
    Returns True if a collision was handled this frame.
    """
    dx = puck.x - paddle.x
    dy = puck.y - paddle.y
    distance = (dx ** 2 + dy ** 2) ** 0.5
    min_dist = puck.radius + paddle.radius
    if distance < min_dist:
        # Avoid division by zero when centres overlap exactly
        if distance == 0:
            dx, dy, distance = 1.0, 0.0, 1.0
        # Unit normal pointing from paddle centre toward puck
        centre
        nx = dx / distance
        ny = dy / distance
```

```

        # Reflect velocity:  $v' = v - 2(v \cdot n)n$ 
        dot = puck.vx * nx + puck.vy * ny
        puck.vx -= 2 * dot * nx
        puck.vy -= 2 * dot * ny
        # Push puck just outside the paddle so it can't collide
again    overlap = min_dist - distance
        puck.x += nx * overlap
        puck.y += ny * overlap
        return True
    return False

```

Key improvements:

- 1 **Proper angle-based reflection** · computes the collision normal (vector from paddle center to puck center), then reflects the velocity using the standard formula $v' = v - 2(v \cdot n)n$.
 - 2 **Overlap separation** · after collision, the puck is pushed out along the normal so it's exactly outside the paddle, preventing it from getting stuck inside.
 - 3 **Edge case handling** · if centers overlap exactly (distance = 0), a default normal is used to avoid division by zero.
-

Prompt 3: Task 2 · Implement Match Scoring

User:

task1 looks good lets start with task2

Assistant:

Implemented match scoring in `game_engine.py`.

Changes made

- 1 **Score variables** · Added `player_score` and `computer_score`, initialized to 0 in `__init__`:

```

self.player_score = 0
self.computer_score = 0

```

- 2 **Score updates on goal** · In `handle_goals()`:

- Puck enters **left goal** (player's side) -> `computer_score += 1`
- Puck enters **right goal** (computer's side) -> `player_score += 1`

```

def _handle_goals(self):
    if self.puck.x - self.puck.radius < MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self.computer_score += 1
            self._reset_puck()
        else:
            self.puck.x = MARGIN + self.puck.radius
            self.puck.vx = -self.puck.vx
    elif self.puck.x + self.puck.radius > WIDTH - MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:

```

```

        self.player_score += 1
        self._reset_puck()
    else:
        self.puck.x = WIDTH - MARGIN - self.puck.radius
        self.puck.vx = -self.puck.vx

```

- 3 **Score display** · `draw()`, both scores are rendered at the top of the screen:

```

renderer.draw_text(surface, font, f"Player: {self.player_score}",
(WIDTH // 4 - 60, 2))
renderer.draw_text(surface, font, f"Computer:
{self.computer_score}", (3 * WIDTH // 4 - 70, 2))

```

Prompt 4: Task 3 · Implement 30-Second Match Timer

User:

looks great the task3 is perfect... lets start with task3

Assistant:

Implemented the 30-second match timer in `game_engine.py`.

Changes made

- 1 `MATCH_DURATION = 30` · 30-second match constant.
- 2 **Timer state** · `start_ticks` captures the match start time via `pygame.time.get_ticks()`, and `game_over` flag tracks whether the match has ended:

```

self.start_ticks = pygame.time.get_ticks()
self.game_over = False

```

- 3 `time_remaining()` **helper** · Computes seconds left:

```

def time_remaining(self):
    elapsed = (pygame.time.get_ticks() - self.start_ticks) / 1000.0
    return max(0, MATCH_DURATION - elapsed)

```

- 4 `update()` **guards** · If `game_over` is `True`, returns immediately. If `time_remaining()` hits 0, sets `game_over = True`:

```

def update(self):
    if self.game_over:
        return
    if self.time_remaining() <= 0:
        self.game_over = True
        return
    # ... rest of game logic

```

- 5 **Timer display** · Shows countdown at top center of screen:

```

remaining = int(self.time_remaining()) + 1 if not self.game_over
else 0

```

```
renderer.draw_text(surface, font, f"Time: {remaining}s", (WIDTH // 2 - 40, 2))
```

6 **End-of-match result** · When game is over, displays a centered banner:

```
if self.game_over:
    if self.player_score > self.computer_score:
        renderer.draw_banner(surface, font, "Player Wins!")
    elif self.computer_score > self.player_score:
        renderer.draw_banner(surface, font, "Computer Wins!")
    else:
        renderer.draw_banner(surface, font, "Draw!")
```

Prompt 5: Task 4 · Implement Puck Reset After Scoring

User:

looks great the task3 is perfect... lets start with task4

Assistant:

Fixed the puck reset in `game_engine.py`. One-line fix.

What changed

Before: `vx = 0, vy = 0` -> puck sat motionless at center forever after a goal

```
def _reset_puck(self):
    self.puck.x, self.puck.y = WIDTH / 2, HEIGHT / 2
    self.puck.vx = 0
    self.puck.vy = 0
```

After: calls `_launch_puck()` -> puck gets a random speed and direction, play resumes immediately

```
def _reset_puck(self):
    self.puck.x, self.puck.y = WIDTH / 2, HEIGHT / 2
    self._launch_puck()
```

Summary of All Tasks

Task	File	Status	Description
Task 1	<code>collisions.py</code>	Done	Proper angle-based reflection + overlap separation
Task 2	<code>game_engine.py</code>	Done	Score tracking, updates on goal, displayed on screen
Task 3	<code>game_engine.py</code>	Done	30s countdown, game stops at 0, winner/draw banner
Task 4	<code>game_engine.py</code>	Done	Re-launches puck with random velocity after goal